## Propriedades da árvore rubro-negra

A árvore rubro-negra é uma árvore binária de busca que se mantém balanceada por conta de uma regra de cores: cada nó é vermelho ou preto, e o que se exige é que cinco propriedades continuem valendo depois de qualquer inserção ou remoção. A raiz é sempre preta, as folhas vazias (NIL) também são pretas, um nó vermelho só pode ter filhos pretos, e todo caminho de um nó até as folhas abaixo dele passa pela mesma quantidade de nós pretos, o que é chamado de altura negra.

Em vista disso, o caminho mais longo da árvore nunca chega a ser mais que o dobro do mais curto, já que o mais curto pode ter só nós pretos e o mais longo precisa alternar entre pretos e vermelhos. Nisso a altura fica limitada a cerca de 2 log n, e busca, inserção e remoção custam O(log n) no pior caso, diferente de uma árvore de busca comum, que degenera para O(n) se receber chaves já ordenadas.

## Inserção

A inserção começa como em qualquer árvore de busca: desce-se comparando as chaves até achar a posição vazia e o novo nó é colocado ali, sempre pintado de vermelho. A escolha do vermelho é proposital, pois adicionar um nó preto alteraria a altura negra de um caminho inteiro, enquanto um nó vermelho só pode quebrar a regra de que dois vermelhos não ficam seguidos. Se o pai for preto, nada mais precisa ser feito (e se o nó for a raiz, basta pintá-lo de preto).

O problema aparece quando o pai também é vermelho, e o que decide o conserto é a cor do tio, ou seja, o irmão do pai. Se o tio for vermelho, pai e tio são recoloridos de preto, o avô vira vermelho e o mesmo teste é repetido a partir do avô, podendo subir até a raiz. Se o tio for preto, entram as rotações: quando o novo nó está "dobrado" em relação ao avô (filho direito de um pai esquerdo, ou o contrário), faz-se primeiro uma rotação no pai para deixá-lo "reto"; em seguida, com o nó reto, faz-se uma rotação no avô e trocam-se as cores de pai e avô, o que encerra o processo. Citando um exemplo, ao inserir as chaves 10, 20 e 30 nessa ordem, o 30 entra vermelho como filho de um pai vermelho (o 20) e com tio NIL, que é preto, e como o nó está reto em relação ao avô basta uma rotação à esquerda no 10, deixando o 20 como raiz preta com 10 e 30 vermelhos. No total, a inserção faz no máximo duas rotações.

## Remoção

A remoção também parte do procedimento da árvore de busca comum: se o nó tem no máximo um filho, ele é substituído pelo filho, e se tem dois, copia-se o valor do sucessor (o menor da subárvore direita) e remove-se o sucessor no lugar, que cai no caso anterior. O que importa para o balanceamento é a cor do nó que foi de fato retirado da estrutura. Se ele era vermelho, nenhuma propriedade é afetada. Se era preto, o caminho que passava por ele perdeu um nó preto, e quem o substituiu passa a carregar um "preto extra" que precisa ser resolvido; caso o substituto seja vermelho, basta pintá-lo de preto e acabou.

Quando o substituto também é preto, o conserto depende do irmão dele (w) e dos sobrinhos. Se w for vermelho, w vira preto, o pai vira vermelho e faz-se uma rotação no pai, transformando o problema em um dos casos seguintes com irmão preto. Se w é preto e os dois filhos de w são pretos, w é pintado de vermelho e o preto extra sobe para o pai, repetindo-se o teste a partir dele. Se w é preto, o sobrinho mais distante é preto e o mais próximo é vermelho, recolore-se e rotaciona-se w para cair no último caso. Por fim, se w é preto e o sobrinho distante é vermelho, w assume a cor do pai, o pai e o sobrinho distante ficam pretos e uma rotação no pai devolve o equilíbrio e termina o processo. A remoção usa no máximo três rotações, e as recolorações podem subir até a raiz, o que mantém tudo em O(log n).